

TDV 2215 Terminal CR/LF Processing Analysis

CR (Carriage Return, 0x0D) Processing

CR Dispatch Entry

Location: Control character dispatch table entry for 0x0D

Table Base: 0x2421

CR Index: 0x0D & 0x1F = 13

Table Offset: $13 \times 2 = 26 = 0x1A$

CR Entry Address: $0x2421 + 0x1A = \mathbf{0x243B}$

CR Handler Flow

```
assembly

; Dispatch table entry for CR (0x0D)
; At 0x243B: 16-bit pointer to actual CR handler

; The actual CR handler I found:
ram:2477: MVI A,0xd      ; Load CR (0x0D) into A register
ram:2479: JMP 0x2531     ; Jump to standard character handler

; Standard character handler:
ram:2531: CALL 0x2557    ; Call terminal_character_output

; terminal_character_output calls escape_sequence_parser
; which eventually calls control_character_handler
```

Direct CR Processing

Function: `reset_cursor_to_line_start` at **0x006a** → **0x134c**

```
assembly

ram:134c: PUSH AF
ram:134d: SUB A          ; Clear A (set cursor column to 0)
ram:134e: OUT (0x3c)     ; Output to display control port
ram:1350: STA (0x5e18)    ; Store cursor column = 0
ram:1353: INR A          ; Increment A to 1
ram:1354: STA (0x5e16)    ; Store next cursor position = 1
ram:1357: POP AF
ram:1358: RET
```

What CR Does:

1. Sets cursor column to 0 (beginning of current line)
2. Outputs control signal to display port 0x3C
3. Updates cursor position memory at 0x5e18 (current) and 0x5e16 (next)

LF (Line Feed, 0x0A) Processing

LF Dispatch Entry

Location: Control character dispatch table entry for 0x0A

Table Base: 0x2421

LF Index: 0x0A & 0x1F = 10

Table Offset: $10 \times 2 = 20 = 0x14$

LF Entry Address: $0x2421 + 0x14 = \mathbf{0x2435}$

Direct LF Processing

Function: `advance_cursor_position` at **0x0076** → `increase_cursor_character` at **0x1382**

```
c

void increase_cursor_character(void)
{
    if (next_cursor_character_address != 0x50) { // If not at end of line
        current_cursor_character_address = next_cursor_character_address;
        next_cursor_character_address = next_cursor_character_address + 1;
        return;
    }
    if (current_cursor_row != 0x19) { // If not at bottom of screen
        update_cursor_addresses(); // Move to next line
        return;
    }
    // At bottom of screen - scroll handling
}
```

What LF Does:

1. If not at end of line (column < 0x50): advance cursor one position
2. If at end of line but not bottom: move to next line (calls `update_cursor_addresses`)
3. If at bottom of screen: handle scrolling

Direct Handler Locations Found

CR Handler Calls

- **0x29bf:** `CALL 0x006a` (reset_cursor_to_line_start)

- **0x2adc:** `CALL 0x006a` (reset_cursor_to_line_start)

LF Handler Calls

- **0x29db:** `CALL 0x0076` (advance_cursor_position)
- **0x2a66:** `CALL 0x0076` (advance_cursor_position)

Debug Points for C# Hardware Emulation

1. Character Dispatch Verification

Monitor these memory locations to verify your C# emulation sends characters to the correct handlers:

Dispatch Table Pointers:

- **0x243B:** Should contain 16-bit address of CR handler (likely 0x29bf)
- **0x2435:** Should contain 16-bit address of LF handler (likely 0x29db)

2. Character Processing Debug

Memory Location 0x5812: Current character being processed

- Set breakpoint when this equals **0x0D** (CR) or **0x0A** (LF)
- Verify the character reaches the terminal firmware

3. CR Processing Debug

When CR (0x0D) is received:

Expected Call Sequence:

1. Character dispatch → CR handler (0x29bf or similar)
2. `CALL 0x006a` (reset_cursor_to_line_start)
3. `OUT (0x3c)` with value 0x00
4. Memory 0x5e18 set to 0x00 (cursor column = 0)
5. Memory 0x5e16 set to 0x01 (next position = 1)

Debug Points:

- **Port 0x3C output:** Should see 0x00 when CR processed
- **Memory 0x5e18:** Should become 0x00 after CR
- **Memory 0x5e16:** Should become 0x01 after CR

4. LF Processing Debug

When LF (0x0A) is received:

Expected Call Sequence:

1. Character dispatch → LF handler (0x29db or similar)
2. `CALL 0x0076` (advance_cursor_position)
3. Cursor advances to next line or next position

Debug Points:

- **Memory 0x5e18:** Cursor column should advance or reset to 0 if wrapping
- **Current cursor row:** Should increment if at end of line

Critical Finding for Your C# Emulation

The Issue: If your C# code is not properly routing CR and LF characters through the control character dispatch table, they won't reach the correct handlers.

Verification Steps:

1. **Send "ABC\r\nDEF"** to your UART emulation
2. **Monitor 0x5812** - should see: 'A', 'B', 'C', **0x0D**, **0x0A**, 'D', 'E', 'F'
3. **Monitor 0x5e18** after 0x0D - should become 0x00
4. **Monitor cursor row** after 0x0A - should increment

If CR (0x0D) doesn't appear in 0x5812, your C# UART emulation is filtering it out before it reaches the terminal firmware.

Expected Behavior

Correct CR+LF Sequence:

1. **CR (0x0D):** Cursor moves to column 0 of current line
2. **LF (0x0A):** Cursor advances to next line
3. **Result:** Proper newline (cursor at column 0 of next line)

Your Observed Bug: Only LF works, CR is ignored

- **Root Cause:** CR characters not reaching dispatch table in your C# emulation
- **Fix:** Ensure your C# Z80 SIO emulation passes 0x0D characters to the terminal firmware unmodified